

Case Study on the Titanic Dataset

The RMS Titanic was a British passenger ship that set sail from Southampton, England, on April 10, 1912, bound for New York City. It was one of the largest and most luxurious ships of its time, often described as “unsinkable.” The Titanic carried over 2,200 passengers and crew members from various social and economic backgrounds. The voyage ended tragically when the ship struck an iceberg in the North Atlantic Ocean. Today, the Titanic dataset is widely used for data analysis and machine learning projects to explore passenger information and uncover meaningful insights.



```
import seaborn as sns
df=sns.load_dataset('titanic')
```

This code imports the Seaborn visualization library as sns and then loads the classic 'titanic' passenger dataset into a Pandas DataFrame variable named df for data analysis and plotting.

- import seaborn as sns: Makes the Seaborn library available under the shortcut sns.
- df = sns.load_dataset('titanic'): Retrieves the 'titanic' dataset and stores it as a DataFrame in df.

▼ **TITANIC DATASET :**

df

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	

891 rows × 15 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

df.shape

(891, 15)

- First element (891): The number of rows in the DataFrame, representing the number of individual passengers (observations) in the dataset.
- Second element (15): The number of columns in the DataFrame, representing the number of features or variables (e.g., 'survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town', 'alive', 'alone'for each passenger.)

```
df.isna().sum()
```

	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

df.isna().sum() quickly counts the number of missing (null) values in each column of the DataFrame. It's the essential first step to identify which columns need data cleaning (imputation or dropping).

- The null values are :age,embarked,deck,embark_town

Replacing Null Values in Titanic Data

▼ AGE :

```
df['age'].fillna(df['age'].mean(),inplace=True)
```

/tmp/ipython-input-1492264711.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation in place.


```
df['age'].fillna(df['age'].mean(),inplace=True)
```

- df['age'].mean(): This part is executed first. It calculates the arithmetic average (mean) of all the non-missing values currently present in the 'age' column.
- fillna(...): This is the Pandas function that finds all the missing values (NaN) in the df['age'] column.

df

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22.000000	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	
1	1	1	female	38.000000	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.000000	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	
887	1	1	female	19.000000	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	
888	0	3	female	29.699118	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	
889	1	1	male	26.000000	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	
890	0	3	male	32.000000	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	

891 rows × 15 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df['age']=df['age'].astype(int)
```

df

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	
1	1	1	female	38	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	26	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	
3	1	1	female	35	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	
4	0	3	male	35	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	
887	1	1	female	19	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	
888	0	3	female	29	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	
889	1	1	male	26	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	
890	0	3	male	32	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	

891 rows × 15 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         891 non-null    int64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    object
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(1), int64(5), object(5)
memory usage: 80.7+ KB
```

▼ EMBARKED :

```
df['embarked'].fillna(df['embarked'].mode()[0],inplace=True)
```

/tmp/ipython-input-1024298632.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation on the original Dataframe.

```
df['embarked'].fillna(df['embarked'].mode()[0],inplace=True)
```

df

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	
1	1	1	female	38	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	26	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	
3	1	1	female	35	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	
4	0	3	male	35	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	
887	1	1	female	19	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	
888	0	3	female	29	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	
889	1	1	male	26	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	
890	0	3	male	32	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	

891 rows × 15 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

DECK :

```
df['deck'].fillna(df['deck'].mode()[0],inplace=True)
```

/tmp/ipython-input-3365195351.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation on the original Dataframe.

```
df['deck'].fillna(df['deck'].mode()[0],inplace=True)
```



EMBARK_TOWN :

```
df['embark_town'].fillna(df['embark_town'].mode()[0],inplace=True)
```

/tmp/ipython-input-751922478.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation on the original Dataframe.

```
df['embark_town'].fillna(df['embark_town'].mode()[0],inplace=True)
```

```
df.isna().sum()
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
deck	0
embark_town	0
alive	0
alone	0

dtype: int64

So we replaced every single missing entry in the 'age', 'embarked','deck','embrak_town',column with the calculated mean age, achieving the goal of making the data complete and ready for mathematical analysis or machine learning models.

Queries for Titanic Dataset :

df.head()

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22	1	0	7.2500	S	Third	man	True	C	Southampton	no	False	
1	1	1	female	38	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	
2	1	3	female	26	0	0	7.9250	S	Third	woman	False	C	Southampton	yes	True	
3	1	1	female	35	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	
4	0	3	male	35	0	0	8.0500	S	Third	man	True	C	Southampton	no	True	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

TOTAL PASSENGERS ARE TRAVELLED IN TITANIC

df.value_counts().sum()

np.int64(891)

->The number 891 represents the total number of passengers in your DataFrame.

HOW MANY MALE AND FEMALES ARE TRAVELLED

df.groupby('sex')['sex'].count()

sex	
sex	
female	314
male	577

dtype: int64

->Here the Females are 314 and Males are 577

TOTAL SURVIVED PASSENGERS


```
df['survived'].sum()
```

```
np.int64(342)
```

->The total numbers of passengers who was survived is 342 out of 891 passengers

How many males and females survived

```
df.groupby('sex')['survived'].sum()
```

	survived
sex	
female	233
male	109

dtype: int64

-> The number of females are survived was 233 and male was 109 .

How many passengers survived from each port (Embarked)

In Titanic has three embarked(ports) : Cherbourg(C),Queenstown(Q),Southampton(S)

```
df.groupby('embark_town')['survived'].sum()
```

	survived
embark_town	
Cherbourg	93
Queenstown	30
Southampton	219

dtype: int64

In Titanic has three embarked(ports) : Cherbourg(C),Queenstown(Q),Southampton(S)

Cherbourg(C) : 93

Queenstown(Q): 30

✓ WHICH DECK HAS MOST PASSENGERS

```
t=df[ 'deck' ].mode()  
t
```

deck	
0	C

dtype: category

- The result shows that Deck C appears most often, indicating that the majority of passengers were accommodated on this deck.
- The mode function helps find the most common category in a column, and in this case, it clearly shows that Deck C had the largest concentration of passengers compared to other decks on the ship.

✓ TO CHECK DECK WISE PASSENGERS

```
df.groupby('deck')['deck'].count()
```

```
/tmp/ipython-input-3796701300.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to r  
df.groupby('deck')['deck'].count()
```

deck	
A	15
B	47
C	747
D	33
E	32
F	13
G	4

dtype: int64

- The code df.groupby('deck')['deck'].count() counts the number of entries for every deck category.

- The output shows that Deck C had the highest number of passengers with 747, followed by Deck B with 47, Deck D with 33, Deck E with 32, Deck A with 15, Deck F with 13, and Deck G with 4 passengers.
- This clearly indicates that Deck C accommodated the majority of passengers on the Titanic, while Deck G had the least.

How many passengers are travelled with sibsp

✓ Sibsp :The number of siblings or spouses a passenger had on board the Titanic.

```
df.groupby('sibsp')['sibsp'].count()
```

sibsp	
sibsp	
0	608
1	209
2	28
3	16
4	18
5	5
8	7

dtype: int64

The command counts how many passengers traveled with 0, 1, 2, etc., siblings or spouses aboard. It groups the data by the 'sibsp' column and then counts the size of each resulting group.

✓ What is the average fare by passenger class

```
avg_fare_class = df.groupby('pclass')['fare'].mean()
print(round(avg_fare_class,2))
```

pclass	
1	84.15
2	20.66
3	13.68
Name: fare, dtype: float64	

- Class 1: The average fare for passengers in the first class is approximately 84.15.
- Class 2: The average fare for passengers in the second class is approximately 20.66.

- Class 3: The average fare for passengers in the third class is approximately 13.68.

OVERVIEW :

Data Overview :

- The dataset contains 891 passengers and 15 columns.
- Missing values in Age, Deck, and Embarked columns were handled (imputed with the mean or mode).

Demographic & Travel:

- Gender: 577 Males and 314 Females.
- Alone: 537 passengers were traveling alone, while 354 were not alone.
- Fare: The average fare shows a clear class disparity: Class 1 (84.15), Class 2 (20.66), and Class 3 (13.68).
- Deck: The majority of recorded passengers were on Deck C (747).

INSIGHTS :

Gender Bias in Survival: The "women and children first" protocol was strongly evident; Females (233) significantly outnumbered Male (109) survivors, accounting for 68% of the total who lived.

Economic Inequality: Average fares reflected class status, with First Class (84.15)*payingdramaticallymorethanThirdClass*(13.68).

Deck Concentration: The majority of passengers with recorded cabin data were concentrated on Deck C (747).

Most Travelers Alone: More passengers traveled alone (537) than with family (354).

Survival by Port: The largest number of survivors embarked from Southampton (219).

RECOMMENDATIONS :

Safety Equity: Mandate that safety access and procedures are independent of ticket price/class to ensure universal safety and equitable survival probability for all passengers.

Targeted Communication: Structure safety briefings to address the majority who travel alone and ensure communications are effective for all embarkation groups (Southampton, Cherbourg, Queenstown).

Modernize Protocol: Move beyond historical biases (like "women first") to implement objective, needs-based evacuation priorities (e.g., children, disabled) that are efficient for all demographics.

